

PROYECTO CATÁLOGO DE TIENDA AVANZADO EN JAVA

Análisis del código, funcionalidades, diseño y polimorfismo

1. DESCRIPCIÓN GENERAL DEL PROYECTO

El programa se llama *CatalogoTiendaAvanzado* y es una aplicación gráfica desarrollada en Java utilizando la librería Swing. Su función principal es simular una tienda virtual de videojuegos o productos digitales, donde el usuario puede visualizar diferentes productos y canjearlos utilizando puntos.

La aplicación permite:

- Visualizar diferentes productos y sus costos.
- Seleccionar un producto del catálogo.
- Elegir una categoría.
- Activar o desactivar notificaciones, envíos al correo.
- Seleccionar entrega digital o física.
- Canjear productos utilizando un sistema de puntos.
- Visualizar iconos generados dinámicamente.
- Recibir mensajes de selección y confirmación.
- Demostrar el uso del polimorfismo en programación orientada a objetos.

2. IMPORTACIONES Y LIBRERÍAS

`import javax.swing.*;` permite importar los principales componentes de Swing, utilizados para crear la interfaz gráfica. Entre ellos se encuentran JFrame, JButton, JLabel, JPanel, JComboBox y JOptionPane.

`import javax.swing.border.TitledBorder;` permite utilizar bordes con títulos, como el borde denominado ZONA DE OPCIONES.

`import java.awt.*;` proporciona herramientas para el diseño gráfico, como Color, Font, Dimension, Graphics2D y GradientPaint.

`import java.awt.image.BufferedImage;` permite crear imágenes directamente mediante código.

`import java.util.HashMap;` e `import java.util.Map;` permiten relacionar el nombre de cada producto con su respectivo icono.

3. JERARQUÍA POLIMÓRFICA

La primera parte importante del programa corresponde a la programación orientada a objetos. Se crea una clase abstracta llamada *ItemTienda*, que representa un producto especial de la tienda.

3.1 Clase abstracta *ItemTienda*

La clase abstracta contiene dos atributos protegidos: nombre, que almacena el nombre del producto, y costo, que almacena su precio en puntos.

Su constructor recibe el nombre y el costo para inicializar los objetos. También contiene los métodos `getNombre()` y `getCosto()`, que permiten obtener estos valores.

El elemento más importante es el método abstracto `aplicarEfectoEspecial()`. Al ser abstracto, las clases hijas están obligadas a implementar su propia versión del método.

3.2 Clase ItemAvatar

La clase `ItemAvatar` hereda de `ItemTienda` mediante la palabra `extends`. Además de nombre y costo, contiene el atributo `tipoRara`, que representa el nivel de rareza del avatar.

El constructor utiliza `super` (nombre, costo) para llamar al constructor de la clase padre. Posteriormente guarda el tipo de rareza.

La anotación `@Override` indica que el método `aplicarEfectoEspecial()` está siendo sobrescrito. En esta clase, el efecto consiste en desbloquear una skin de avatar e informar su nivel de rareza.

3.3 Clase ItemPase

La clase `ItemPase` también hereda de `ItemTienda`. Posee el atributo `diasPase`, que representa la duración del pase.

Esta clase implementa `aplicarEfectoEspecial()` de una forma diferente: en lugar de desbloquear una skin, otorga un pase académico especial durante una determinada cantidad de días.



4. CLASE PRINCIPAL: CatalogoTiendaAvanzado

La clase principal del programa es `CatalogoTiendaAvanzado`. Esta clase hereda de `JFrame`, por lo que representa la ventana principal de la aplicación.

Dentro de esta clase se declaran los componentes principales de la interfaz:

- `JComboBox`: permite seleccionar una categoría.

- JCheckBox: permite activar o desactivar las notificaciones.
- JRadioButton: permite seleccionar entre entrega digital o física.
- ButtonGroup: evita que ambas opciones de entrega estén seleccionadas al mismo tiempo.
- JPanel: organiza las diferentes zonas de la interfaz.
- JLabel: muestra los puntos del usuario.
- ItemTienda[]: almacena objetos para la demostración del polimorfismo.
- Map<String, ImageIcon>: relaciona cada producto con su icono.

5. SISTEMA DE PUNTOS Y SELECCIÓN

El usuario inicia con 1000 puntos. La variable puntosUsuario almacena la cantidad disponible.

La variable productoSeleccionado guarda el nombre del producto elegido y comienza con el valor Ninguno. La variable costoSeleccionado almacena el costo del producto seleccionado y comienza con el valor 0.

Cuando el usuario selecciona un producto, el programa guarda automáticamente su nombre y su costo. Cuando realiza el canje, el costo se descuenta de los puntos disponibles.

6. CONFIGURACIÓN DE LA VENTANA

El constructor CatalogoTiendaAvanzado() construye toda la interfaz gráfica.

- ejecutarDemostracionPolimorfismo(): ejecuta el ejemplo de polimorfismo en la consola.
- inicializarImagenesGeneradas(): crea los iconos de los productos.
- setTitle(): establece el título de la ventana.
- setSize(650, 550): establece el tamaño de la ventana.
- setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE): permite cerrar completamente el programa.
- setLocationRelativeTo(null): centra la ventana en la pantalla.
- setLayout(new BorderLayout(10, 10)): organiza la ventana en zonas y añade espacios entre componentes.

7. DISEÑO Y PALETA DE COLORES

El proyecto utiliza una temática oscura inspirada en videojuegos de acción y suspenso. La combinación de colores busca crear una interfaz llamativa, moderna y con apariencia gamer.

Elemento	Color	Función
Fondo principal	Gris oscuro / Negro	Crear una temática oscura.

Panel superior	Vino oscuro	Separar visualmente la zona de opciones.
Botones	Vino intenso / Rojo	Destacar las acciones disponibles.
Textos	Blanco hueso	Mejorar la lectura sobre fondos oscuros.
Puntos	Verde brillante	Resaltar la cantidad disponible.

8. FUENTES UTILIZADAS

Se utiliza la fuente Impact en los títulos para proporcionar una apariencia más fuerte y relacionada con videojuegos.

También se utiliza Segoe UI en negrita para los textos y componentes, ya que ofrece una lectura clara y limpia.

9. ESTRUCTURA DE LA INTERFAZ

La ventana utiliza BorderLayout, que divide la interfaz en tres zonas principales:

9.1 Panel Norte: Zona de opciones

Utiliza FlowLayout para organizar sus componentes horizontalmente. Contiene el indicador de puntos, el filtro de categorías, la opción de recibir alertas y los botones para elegir entre entrega digital o física.

9.2 Panel Central: Catálogo de productos

Utiliza GridLayout(3, 2, 12, 12), lo que organiza los seis productos en una cuadrícula de tres filas y dos columnas. Cada producto se representa mediante un botón con nombre, costo e icono.

9.3 Panel Sur: Canje

Contiene el botón principal CANJEAR ÍTEM. Este botón procesa la selección del usuario y realiza las validaciones necesarias antes de descontar los puntos.

10. CREACIÓN AUTOMÁTICA DE PRODUCTOS

Los productos y sus costos se almacenan en dos arreglos. Un arreglo contiene los nombres y otro contiene los costos correspondientes.

Posteriormente se utiliza un ciclo for para recorrer ambos arreglos y crear automáticamente los botones. Esto evita tener que programar manualmente cada producto.

Los productos disponibles son:

- Skin de Avatar – 100 puntos.
- Pase de Tarea – 200 puntos.
- Cursor Personalizado – 50 puntos.
- Insignia Dorada – 150 puntos.
- Fondo Exclusivo – 80 puntos.
- Bonus Sorpresa – 300 puntos.

11. EVENTOS Y SELECCIÓN DE PRODUCTOS

Cada botón posee un ActionListener. Este componente detecta cuándo el usuario realiza un clic.

Cuando se selecciona un producto, el programa:

- Guarda el nombre en productoSeleccionado.
- Guarda el costo en costoSeleccionado.
- Obtiene el icono correspondiente.
- Muestra una ventana emergente mediante JOptionPane con la información del producto.

12. SCROLLPANE

El catálogo se encuentra dentro de un JScrollPane. Esto permite agregar desplazamiento en caso de que en el futuro aumente la cantidad de productos disponibles.

13. FUNCIONAMIENTO DEL CANJE

Cuando el usuario presiona CANJEAR ÍTEM, se ejecutan varias validaciones.

13.1 Validación de producto seleccionado

Si productoSeleccionado tiene el valor Ninguno, el sistema muestra una advertencia indicando que primero debe seleccionarse un producto.

13.2 Validación de puntos

El programa compara puntosUsuario con costoSeleccionado. Si los puntos disponibles son menores que el costo, se muestra un mensaje de error.

13.3 Canje exitoso

Si todas las condiciones son correctas, el programa descuenta el costo del producto, actualiza visualmente los puntos y obtiene las opciones seleccionadas por el usuario.

Finalmente se muestra una ventana de confirmación con el producto, puntos gastados, puntos restantes, categoría, tipo de entrega y estado de las notificaciones.

Después del canje, `productoSeleccionado` vuelve a `Ninguno` y `costoSeleccionado` vuelve a 0.

14. HASHMAP Y MAPA DE IMÁGENES

El programa utiliza un `HashMap` para relacionar el nombre de cada producto con su imagen.

La relación funciona de la siguiente manera:

- Skin de Avatar → SKIN
- Pase de Tarea → PASS
- Cursor Personalizado → CUR
- Insignia Dorada → BADGE
- Fondo Exclusivo → BG
- Bonus Sorpresa → BOX

15. GENERACIÓN DINÁMICA DE ICONOS

El método `crearIconoGrafico()` genera una imagen automáticamente mediante código.

Primero se establece un tamaño de 48 por 48 píxeles. Después se crea un `BufferedImage` con soporte para transparencia.

`Graphics2D` permite dibujar dentro de la imagen. Se activa el suavizado gráfico para mejorar la apariencia visual.

Posteriormente se crea un `GradientPaint` que combina un color gris oscuro con un color secundario. Se dibuja un rectángulo con bordes redondeados, se añade un borde y finalmente se coloca una etiqueta como SKIN, PASS o BOX.

Al finalizar, `g2d.dispose()` libera los recursos gráficos y la imagen se convierte en `ImageIcon` para poder utilizarla en los botones de Swing.

16. POLIMORFISMO APLICADO

El polimorfismo es uno de los conceptos principales utilizados en este proyecto. La estructura está formada por una clase abstracta llamada `ItemTienda` y dos clases hijas: `ItemAvatar` e `ItemPase`.

La clase padre declara el método abstracto `aplicarEfectoEspecial()`. Cada clase hija implementa este mismo método de forma diferente:

- `ItemAvatar`: desbloquea una skin de avatar e informa su rareza.
- `ItemPase`: otorga un pase durante una cantidad determinada de días.

El arreglo `catalogoEspecial` es de tipo `ItemTienda`, pero puede almacenar objetos `ItemAvatar` e `ItemPase`. Al recorrer el arreglo y ejecutar `item.aplicarEfectoEspecial()`, Java identifica automáticamente el tipo real del objeto y ejecuta el comportamiento correspondiente.

Por esta razón, el programa demuestra polimorfismo: diferentes objetos responden al mismo método de formas distintas.

17. MÉTODO MAIN

El método main es el punto de inicio del programa.

SwingUtilities.invokeLater() permite iniciar la interfaz gráfica de forma segura dentro del hilo correspondiente de Swing.

Finalmente, new CatalogoTiendaAvanzado().setVisible(true) crea la ventana y la muestra en pantalla.

18. PRINCIPALES CONCEPTOS UTILIZADOS

- Programación Orientada a Objetos.
- Clase abstracta.
- Herencia.
- Polimorfismo.
- Sobrescritura de métodos mediante @Override.
- Arreglos.
- HashMap.
- Eventos con ActionListener.
- Interfaces gráficas con Java Swing.
- Validación de datos.
- Métodos auxiliares.
- Bucles for.
- Generación de gráficos con Graphics2D.

19. CONCLUSIÓN

CatalogoTiendaAvanzado es un proyecto que combina programación orientada a objetos e interfaces gráficas en Java. Su desarrollo permite aplicar conceptos importantes como herencia, polimorfismo, clases abstractas, eventos, validaciones, arreglos y estructuras de datos.

Además, la interfaz ofrece una experiencia visual organizada mediante diferentes layouts y una temática personalizada. El proyecto demuestra cómo Java Swing puede utilizarse para desarrollar aplicaciones interactivas que integren lógica de programación y diseño gráfico.

Código:

```
import javax.swing.*;  
import javax.swing.border.TitledBorder;  
import java.awt.*;  
import java.awt.image.BufferedImage;
```

```

import java.util.HashMap;
import java.util.Map;

// =====
// 1. JERARQUÍA POLIMÓRFICA (POLIMORFISMO)
// =====
abstract class ItemTienda {
    protected String nombre;
    protected int costo;

    public ItemTienda(String nombre, int costo) {
        this.nombre = nombre;
        this.costo = costo;
    }

    public String getNombre() { return nombre; }
    public int getCosto() { return costo; }

    // Método abstracto (Comportamiento Polimórfico)
    public abstract void aplicarEfectoEspecial();
}

class ItemAvatar extends ItemTienda {
    private String tipoRareza; // Legendario, Raro, etc.

    public ItemAvatar(String nombre, int costo, String tipoRareza) {
        super(nombre, costo);
        this.tipoRareza = tipoRareza;
    }

    @Override
    public void aplicarEfectoEspecial() {
        System.out.println("Ítem: " + nombre + " | Costo: " + costo + " pts");
        System.out.println("Desbloqueando skin de avatar tipo: " + tipoRareza);
    }
}

class ItemPase extends ItemTienda {
    private int diasPase;

    public ItemPase(String nombre, int costo, int diasPase) {
        super(nombre, costo);
        this.diasPase = diasPase;
    }

    @Override
    public void aplicarEfectoEspecial() {
        System.out.println("Ítem: " + nombre + " | Costo: " + costo + " pts");
    }
}

```

```

        System.out.println("Otorgando pase académico especial por " + diasPase + "
días.");
    }
}

// =====
// 2. CLASE PRINCIPAL SWING
// =====
public class CatalogoTiendaAvanzado extends JFrame {
    // Componentes de selección
    private JComboBox<String> cmbCategorias;
    private JCheckBox chkNotificaciones;
    private JRadioButton rbtDigital, rbtFisico;
    private ButtonGroup grupoEnvio;
    private JPanel panelFiltros, panelCentro, panelSur;

    // Control de puntos y selección
    private int puntosUsuario = 1000;
    private JLabel lblPuntos;
    private String productoSeleccionado = "Ninguno";
    private int costoSeleccionado = 0;
    private String detalleAvatar = ""; // Almacena el nivel de rareza si es Avatar

    // Colección polimórfica y mapa de imágenes procedimentales
    private ItemTienda[] catalogoEspecial;
    private Map<String, ImageIcon> mapaImagenes;

    public CatalogoTiendaAvanzado() {
        // --- EJECUCIÓN DE DEMOSTRACIÓN DE POLIMORFISMO EN CONSOLA ---
        ejecutarDemostracionPolimorfismo();

        // Generar iconos dinámicos para los productos
        inicializarImagenesGeneradas();

        setTitle("GameStore Escolar - Catálogo Avanzado");
        setSize(650, 550);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        // --- PALETA DE COLORES (ESTILO ZOMBIE/ACCIÓN) ---
        Color colorFondoOscuro = new Color(25, 25, 25); // Gris oscuro/Negro
        Color colorVinoBase = new Color(80, 15, 25); // Vino oscuro
        Color colorVinoBoton = new Color(130, 25, 40); // Vino intenso
        Color colorTexto = new Color(230, 230, 230); // Blanco hueso
        Color colorVerdePuntos = new Color(80, 200, 120); // Verde neón

        // --- FUENTES ESTÁNDAR DE JAVA ---

```

```

Font fuenteTitulo = new Font("Impact", Font.PLAIN, 16);
Font fuenteTexto = new Font("Segoe UI", Font.BOLD, 12);

getContentPane().setBackground(colorFondoOscuro);

// --- PANEL NORTE: FILTROS + PUNTOS DEL JUGADOR ---
panelFiltros = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 8));
panelFiltros.setBackground(colorVinoBase);

TitledBorder bordeFiltros = BorderFactory.createTitledBorder(
    BorderFactory.createLineBorder(new Color(180, 40, 50), 2),
    " ZONA DE OPCIONES ",
    TitledBorder.LEFT, TitledBorder.TOP, fuenteTitulo, colorTexto);
panelFiltros.setBorder(bordeFiltros);

// Indicador de Puntos
lblPuntos = new JLabel("Puntos: " + puntosUsuario + " PTS");
lblPuntos.setFont(fuenteTitulo);
lblPuntos.setForeground(colorVerdePuntos);
panelFiltros.add(lblPuntos);

// Categorías
cmbCategorias = new JComboBox<>(new String[]{"Todas las categorías",
"Avatars", "Pases", "Accesorios"});
cmbCategorias.setFont(fuenteTexto);
cmbCategorias.setBackground(colorVinoBoton);
cmbCategorias.setForeground(colorTexto);

JLabel lblFiltrar = new JLabel("Filtrar:");
lblFiltrar.setFont(fuenteTexto);
lblFiltrar.setForeground(colorTexto);
panelFiltros.add(lblFiltrar);
panelFiltros.add(cmbCategorias);

// CheckBox Alerta por Correo
chkNotificaciones = new JCheckBox("Recibir alerta por correo");
chkNotificaciones.setFont(fuenteTexto);
chkNotificaciones.setBackground(colorVinoBase);
chkNotificaciones.setForeground(colorTexto);
panelFiltros.add(chkNotificaciones);

// RadioButtons Entrega
rbtDigital = new JRadioButton("Digital", true);
rbtDigital.setFont(fuenteTexto);
rbtDigital.setBackground(colorVinoBase);
rbtDigital.setForeground(colorTexto);

rbtFisico = new JRadioButton("Físico");

```

```

rbtFisico.setFont(fuenteTexto);
rbtFisico.setBackground(colorVinoBase);
rbtFisico.setForeground(colorTexto);

grupoEnvio = new ButtonGroup();
grupoEnvio.add(rbtDigital);
grupoEnvio.add(rbtFisico);

panelFiltros.add(rbtDigital);
panelFiltros.add(rbtFisico);

add(panelFiltros, BorderLayout.NORTH);

// --- PANEL CENTRAL: CATÁLOGO DE ÍTEMS ---
panelCentro = new JPanel(new GridLayout(3, 2, 12, 12));
panelCentro.setBackground(colorFondoOscuro);
panelCentro.setBorder(BorderFactory.createEmptyBorder(10, 15, 10, 15));

String[] productos = {
    "Skin de Avatar", "Pase de Tarea", "Cursor Personalizado",
    "Insignia Dorada", "Fondo Exclusivo", "Bonus Sorpresa"
};
int[] costos = {100, 200, 50, 150, 80, 300};

for (int i = 0; i < productos.length; i++) {
    String nombre = productos[i];
    int costoBase = costos[i];

    JButton btn = new JButton(nombre + " (" + costoBase + " pts)");
    btn.setFont(fuenteTexto);
    btn.setBackground(colorVinoBoton);
    btn.setForeground(colorTexto);
    btn.setFocusPainted(false);

    if (mapaImagenes.containsKey(nombre)) {
        btn.setIcon(mapaImagenes.get(nombre));
        btn.setHorizontalTextPosition(SwingConstants.RIGHT);
    }

    btn.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(new Color(180, 40, 50), 1),
        BorderFactory.createEmptyBorder(10, 10, 10, 10)
    ));

    btn.addActionListener(e -> {
        if (nombre.equals("Skin de Avatar")) {
            // Selección exclusiva de nivel/rareza para el Avatar
            String[] opcionesAvatar = {

```

```

        "Nivel 1: Normal (100 pts)",
        "Nivel 2: Raro (150 pts)",
        "Nivel 3: Ultra Raro (200 pts)",
        "Nivel 4: Legendario (300 pts)"
    };

    String seleccion = (String) JOptionPane.showInputDialog(
        this,
        "Selecciona el Nivel y Rareza de tu Avatar:",
        "Configuración de Avatar",
        JOptionPane.QUESTION_MESSAGE,
        mapaImagenes.get(nombre),
        opcionesAvatar,
        opcionesAvatar[0]
    );

    if (seleccion != null) {
        productoSeleccionado = nombre;
        if (seleccion.contains("Nivel 1")) {
            costoSeleccionado = 100;
            detalleAvatar = "Nivel 1: Normal";
        } else if (seleccion.contains("Nivel 2")) {
            costoSeleccionado = 150;
            detalleAvatar = "Nivel 2: Raro";
        } else if (seleccion.contains("Nivel 3")) {
            costoSeleccionado = 200;
            detalleAvatar = "Nivel 3: Ultra Raro";
        } else if (seleccion.contains("Nivel 4")) {
            costoSeleccionado = 300;
            detalleAvatar = "Nivel 4: Legendario";
        }

        JOptionPane.showMessageDialog(this,
            "Elegiste: " + nombre + "\nRareza: " + detalleAvatar +
            "\nCosto: " + costoSeleccionado + " pts\n\nPresiona 'Canjear Ítem' abajo para
            confirmar la compra.",
            "Ítem Seleccionado",
            JOptionPane.INFORMATION_MESSAGE,
            mapaImagenes.get(nombre));
    }
    } else {
        // Para los demás ítems se mantiene el costo estándar sin
solicitar nivel
        productoSeleccionado = nombre;
        costoSeleccionado = costoBase;
        detalleAvatar = "";

        JOptionPane.showMessageDialog(this,

```

```

        "Elegiste: " + nombre + "\nCosto: " + costoSeleccionado + "
pts\n\nPresiona 'Canjear Ítem' abajo para confirmar la compra.",
        "Ítem Seleccionado",
        JOptionPane.INFORMATION_MESSAGE,
        mapaImagenes.get(nombre));
    }
});

panelCentro.add(btn);
}

JScrollPane scrollPane = new JScrollPane(panelCentro);
scrollPane.setBorder(null);
add(scrollPane, BorderLayout.CENTER);

// --- PANEL SUR: BOTÓN DE CANJE ---
panelSur = new JPanel();
panelSur.setBackground(colorFondoOscuro);

JButton btnCanjear = new JButton("CANJEAR ÍTEM");
btnCanjear.setFont(fuenteTitulo);
btnCanjear.setBackground(new Color(180, 30, 40));
btnCanjear.setForeground(colorTexto);
btnCanjear.setPreferredSize(new Dimension(200, 40));
btnCanjear.setFocusPainted(false);

btnCanjear.addActionListener(e -> {
    if (productoSeleccionado.equals("Ninguno")) {
        JOptionPane.showMessageDialog(this, "Primero selecciona un producto de
la grilla.", "Atención", JOptionPane.WARNING_MESSAGE);
    } else if (puntosUsuario < costoSeleccionado) {
        JOptionPane.showMessageDialog(this, "¡No tienes suficientes puntos
para este ítem!", "Puntos Insuficientes", JOptionPane.ERROR_MESSAGE);
    } else {
        puntosUsuario -= costoSeleccionado;
        lblPuntos.setText("Puntos: " + puntosUsuario + " PTS");

        String tipoEntrega = rbtDigital.isSelected() ? "Digital" : "Físico";
        String alerta = chkNotificaciones.isSelected() ? "Activada" :
"Desactivada";

        String resumen = "=== CONFIRMACIÓN DE CANJE ===\n"
            + "Ítem: " + productoSeleccionado + (detalleAvatar.isEmpty() ?
"" : " (" + detalleAvatar + ")") + "\n"
            + "Puntos gastados: " + costoSeleccionado + " pts\n"
            + "Puntos restantes: " + puntosUsuario + " pts\n"
            + "Categoría: " + cmbCategorias.getSelectedItem() + "\n"
            + "Entrega: " + tipoEntrega + "\n"

```

```

        + "Alerta por correo: " + alerta;

        JOptionPane.showMessageDialog(this, resumen, "¡Canje Exitoso!",
JOptionPane.INFORMATION_MESSAGE, mapaImagenes.get(productoSeleccionado));

        productoSeleccionado = "Ninguno";
        costoSeleccionado = 0;
        detalleAvatar = "";
    }
});

panelSur.add(btnCanjear);
add(panelSur, BorderLayout.SOUTH);
}

// =====
// 3. MÉTODOS AUXILIARES Y POLIMORFISMO
// =====
private void ejecutarDemostracionPolimorfismo() {
    catalogoEspecial = new ItemTienda[]{
        new ItemAvatar("Skin Guerrero Cyber", 150, "Legendario"),
        new ItemPase("Pase de Prórroga", 300, 7)
    };

    System.out.println("--- EJECUTANDO EFECTOS CON POLIMORFISMO ---");
    for (ItemTienda item : catalogoEspecial) {
        item.aplicarEfectoEspecial();
        System.out.println("-----");
    }
}

private void inicializarImagenesGeneradas() {
    mapaImagenes = new HashMap<>();

    mapaImagenes.put("Skin de Avatar", crearIconoGrafico("SKIN", new Color(180,
40, 50)));
    mapaImagenes.put("Pase de Tarea", crearIconoGrafico("PASS", new Color(40, 100,
180)));
    mapaImagenes.put("Cursor Personalizado", crearIconoGrafico("CUR", new
Color(180, 140, 30)));
    mapaImagenes.put("Insignia Dorada", crearIconoGrafico("BADGE", new Color(210,
160, 20)));
    mapaImagenes.put("Fondo Exclusivo", crearIconoGrafico("BG", new Color(100, 40,
150)));
    mapaImagenes.put("Bonus Sorpresa", crearIconoGrafico("BOX", new Color(40, 160,
80)));
}

```

```

    private ImageIcon crearIconoGrafico(String etiqueta, Color colorSecundario) {
        int width = 48;
        int height = 48;
        BufferedImage img = new BufferedImage(width, height,
BufferedImage.TYPE_INT_ARGB);
        Graphics2D g2d = img.createGraphics();

        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        GradientPaint gp = new GradientPaint(0, 0, new Color(40, 40, 40), width,
height, colorSecundario);
        g2d.setPaint(gp);
        g2d.fillRoundRect(2, 2, width - 4, height - 4, 10, 10);

        g2d.setColor(new Color(230, 230, 230));
        g2d.setStroke(new BasicStroke(1.5f));
        g2d.drawRoundRect(2, 2, width - 4, height - 4, 10, 10);

        g2d.setFont(new Font("Impact", Font.PLAIN, 12));
        FontMetrics fm = g2d.getFontMetrics();
        int textX = (width - fm.stringWidth(etiqueta)) / 2;
        int textY = (height + fm.getAscent() - fm.getDescent()) / 2;

        g2d.setColor(Color.WHITE);
        g2d.drawString(etiqueta, textX, textY);

        g2d.dispose();
        return new ImageIcon(img);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new
CatalogoTiendaAvanzado().setVisible(true));
    }
}

```

Catálogo Tienda Avanzado:





